Práctica 01. Compilación frente a interpretación

Datos de la práctica	
Criterios de evaluación	CE1.c, CE1.d y CE1.f del RA1
Duración orientativa	2 sesiones (unos 100 minutos)
Materiales	Visual Studio Code, compilador gcc, Python 3 y JDK de Java (javac y java)
Entregable	Informe con capturas de pantalla y las tablas completadas (PDF o documento digital)

Objetivo

Comprobar **de forma objetiva** las diferencias entre **compilar**, **interpretar** y ejecutar en una **máquina virtual**, usando el mismo programa en tres lenguajes: **C** (compilado), **Python** (interpretado) y **Java** (compilado a bytecode y ejecutado por la JVM).

> **IDEA CLAVE** · Vas a observar tres cosas: **qué ficheros se generan**, **cuándo aparecen los errores** y **cuánto tarda cada programa**. Anota todo lo que veas: con esos datos responderás a las conclusiones finales.

Preparación del entorno

- Crea una carpeta `practica01` con tres subcarpetas: `c`, `python` y `java`.
- Abre la carpeta `practica01` en **Visual Studio Code** (*Archivo → Abrir carpeta*). El panel **Explorador** de la izquierda te mostrará los ficheros que se vayan generando.
- Abre el **terminal integrado** (*Terminal → Nuevo terminal*) y comprueba que las herramientas están instaladas:

```
gcc --version
python --version      (en Linux/macOS: python3 --version)
javac -version
```

Si alguno de los comandos no se reconoce, avisa al profesor antes de continuar.

Parte 1. Programa compilado: C

El programa suma los números del 1 al 200 000 000 y muestra el resultado. Crea el archivo `c/suma.c` con este código:

```
#include <stdio.h>
int main() {
    long long suma = 0;
    printf("Inicio del programa\n");
    for (long long i = 1; i <= 200000000; i++) {
        suma += i;
    }
    printf("La suma es: %lld\n", suma);
    return 0;
}
```

Desde el terminal, sitúate en la carpeta `c` (`cd c`) y recorre el proceso de compilación **paso a paso**:

Paso	Comando	Qué debes observar
1	`gcc -c suma.c`	Se genera `suma.o` (código objeto). Anota su tamaño y comprueba si puedes leerlo en el editor.
2	`gcc suma.o -o suma`	El enlazador genera el ejecutable (`suma` o `suma.exe`). Anota su tamaño.
3	`./suma` (Windows: `.\suma.exe`)	El programa se ejecuta sin necesidad del compilador.

Anota: ¿Qué ficheros hay ahora en la carpeta `c`? ¿Cuál es el código fuente, cuál el código objeto y cuál el ejecutable?

Parte 2. Programa interpretado: Python

Crea el archivo `python/suma.py` con el mismo algoritmo:

```
suma = 0
print("Inicio del programa")
for i in range(1, 200000001):
    suma += i
print("La suma es:", suma)
```

Ejecútalo desde la carpeta `python` con `python suma.py` (en Linux/macOS, `python3 suma.py`).

Anota: ¿Ha aparecido algún fichero nuevo en la carpeta? ¿Hace falta algo instalado en el equipo para poder ejecutarlo?

Parte 3. Lenguaje virtual: Java

Crea el archivo `java/Suma.java`:

```
public class Suma {
    public static void main(String[] args) {
        long suma = 0;
        System.out.println("Inicio del programa");
        for (long i = 1; i <= 200000000L; i++) {
            suma += i;
        }
        System.out.println("La suma es: " + suma);
    }
}
```

Paso	Comando (en la carpeta `java`)	Qué debes observar
1	`javac Suma.java`	Se genera `Suma.class`: el **bytecode**. Anota su tamaño.
2	`java Suma`	La **JVM** ejecuta el bytecode. Fíjate en que se escribe `Suma`, sin extensión.
3	`javap -c Suma` (opcional)	Muestra las instrucciones del bytecode. Compáralas con el código fuente.

Anota: ¿Se parece `Suma.class` a un ejecutable de C? ¿Podrías ejecutarlo con doble clic? ¿Qué necesita el equipo para ejecutarlo?

Parte 4. ¿Cuándo aparecen los errores?

Ahora introduciremos un error a propósito: usar una variable `total` que **no existe**. Crea estos tres archivos, uno en cada carpeta:

`c/error.c`

```c
#include <stdio.h>
int main() {
    printf("Linea 1: todo bien\n");
    printf("Linea 2: todo bien\n");
    printf("Resultado: %d\n", total);
    return 0;
}
```

`python/error.py`

```python
print("Linea 1: todo bien")
print("Linea 2: todo bien")
print("Resultado:", total)
```

`java/Error.java`

```java
public class Error {
    public static void main(String[] args) {
        System.out.println("Linea 1: todo bien");
        System.out.println("Linea 2: todo bien");
        System.out.println("Resultado: " + total);
    }
}
```

Compila y ejecuta cada uno (`gcc error.c -o error`, `python error.py`, `javac Error.java`) y completa la tabla:

Pregunta	C	Python	Java
¿Se muestra «Linea 1: todo bien»?			
¿Se genera ejecutable o `.class`?			
¿En qué momento aparece el error? (antes o durante la ejecución)			
¿Qué dice el mensaje de error?			

Parte 5. ¿Cuánto tarda cada programa?

Vuelve a las tres versiones de `suma` y mide el tiempo de ejecución **tres veces** cada una, para tener un valor medio fiable. Cierra otros programas pesados mientras mides.

Programa	Linux / macOS	Windows (PowerShell)
C	`time ./suma`	`Measure-Command { .\suma.exe }`
Python	`time python3 suma.py`	`Measure-Command { python suma.py }`
Java	`time java Suma`	`Measure-Command { java Suma }`

Programa	Intento 1	Intento 2	Intento 3	Media
C (compilado)				
Python (interpretado)				
Java (JVM)				

Pista: el tiempo que te interesa es el **real** (`real`) o el total que muestra `Measure-Command`.

Parte 6. Conclusiones

Completa la tabla comparativa con lo que **has observado** (no con lo que recuerdes de la teoría):

Aspecto	C	Python	Java
¿Cuándo se traduce el código?			
¿Qué ficheros se generan?			
¿Qué necesita el equipo para ejecutarlo?			
¿Cómo aparecen los errores?			
Tiempo medio observado			

Responde brevemente (3–4 líneas por pregunta):

- ¿Qué diferencia hay entre el código objeto de C y el bytecode de Java?
- ¿Por qué en C el programa con error no llegó a mostrar «Linea 1» y en Python sí?
- ¿A qué crees que se debe la diferencia de tiempos entre C y Python? ¿Y la de Java?
- Si tuvieras que entregar un programa a un cliente sin que instale nada, ¿qué opción elegirías y por qué?
- Si quisieras que el mismo programa funcionara en Windows, Linux y macOS sin recompilar, ¿qué opción elegirías y por qué?

Ampliación (opcional)

- Compila el programa en C con optimización (`gcc -O2 suma.c -o suma_o2`) y vuelve a medir el tiempo. ¿Qué ocurre? ¿Por qué crees que el compilador puede hacerlo?
- Cambia el límite del bucle (por ejemplo, 20 000 000 y 2 000 000 000) y comprueba si la diferencia entre lenguajes se mantiene.

Entregable

Un único informe con: (1) capturas del Explorador de VS Code mostrando los ficheros generados en cada carpeta, (2) capturas de los mensajes de error de la Parte 4, (3) las tres tablas completadas y (4) las respuestas de las conclusiones.

Rúbrica de corrección (sobre 10 puntos)

Criterio de corrección	Puntos
Los tres programas se compilan/ejecutan correctamente y se aportan capturas de los ficheros generados	2
Tabla de errores completa y coherente con lo observado	2
Tiempos medidos tres veces y media calculada correctamente	2
Tabla comparativa completa, basada en la observación	2
Respuestas a las conclusiones razonadas y relacionadas con la teoría (compilador, intérprete, bytecode y JVM)	2
Total	**10**